

Smart Blind Assistant

Pipeline de développement — De zéro à la démo finale

Ce pipeline te guide étape par étape dans l'ordre exact où tu dois développer le projet. Chaque phase a un objectif clair. **Ne passe pas à la suivante tant que la précédente ne marche pas.**

#	Phase	Ce que tu construis	Durée estimée
1	Environnement	Setup Python + Flutter + outils	½ journée
2	Backend Skeleton	FastAPI avec toutes les routes vides	1 journée
3	Computer Vision	Détection obstacles + objets + pièce	3-4 jours
4	NLP	Comprendre les commandes vocales	2 jours
5	Navigation	Graphe maison + calcul chemin + instructions	2 jours
6	SOS	Gestion urgences	½ journée
7	App Flutter	4 écrans + connexion backend	3-4 jours
8	Intégration	Tout brancher ensemble + tests	2 jours
9	Démo	Préparer la présentation	1 journée

PHASE 1 Environnement & Setup

Installer tous les outils avant de coder

■	Installer Python 3.10+	python.org → vérifie avec : <code>python --version</code>
■	Créer un virtualenv	<code>python -m venv venv</code> puis <code>source venv/bin/activate</code>
■	Installer FastAPI + Uvicorn	<code>pip install fastapi uvicorn python-multipart</code>
■	Installer YOLO (Ultralytics)	<code>pip install ultralytics</code> → teste avec un script basique
■	Installer Sentence-Transformers	<code>pip install sentence-transformers</code>
■	Installer Flutter + Dart	flutter.dev → vérifie avec : <code>flutter doctor</code>
■	Installer VS Code + extensions	Python, Flutter, REST Client (pour tester les routes)

■ Critère de validation : tu peux lancer 'uvicorn main:app' et ouvrir flutter doctor sans erreur.

PHASE 2 Backend Skeleton

Créer toutes les routes FastAPI — vides pour l'instant

■	Créer la structure des dossiers	<code>cv/ nlp/ navigation/ safety/ + main.py</code>
■	Créer main.py avec toutes les routes	<code>POST /detect-obstacle /find-object /where-am-i /navigate /sos</code>
■	Chaque route retourne une réponse mock	Ex: <code>return {'status': 'ok', 'result': 'mock'}</code> pour tester
■	Tester toutes les routes	Via navigateur : <code>http://localhost:8000/docs</code> (Swagger auto)

main.py de départ :

```
from fastapi import FastAPI
app = FastAPI()

@app.post("/detect-obstacle")
def detect(): return {"obstacles": []}

@app.post("/navigate")
def navigate(): return {"steps": []}
```

■ Critère : `http://localhost:8000/docs` affiche toutes les routes sans erreur.

PHASE 3 Computer Vision

Brancher YOLO pour la vision en temps réel

■	detector.py — détection obstacles	Prend une image → YOLO → retourne liste d'obstacles + positions
■	finder.py — chercher un objet	Prend image + nom objet → retourne si trouvé + distance estimée
■	localizer.py — identifier la pièce	Détecte les objets présents → compare avec house.json → retourne pièce
■	Brancher dans main.py	Remplacer les mocks par de vrais appels à detector.py / finder.py
■	Tester avec des images fixes	Envoie des photos de test via Swagger avant de brancher la caméra

■ Conseil : commence avec YOLOv8n (nano) — le plus rapide, suffisant pour une démo.

PHASE 4 NLP — Comprendre les commandes

Transformer la voix en intention

■	Créer intents.json	Liste d'exemples pour chaque intention : FIND_OBJECT, NAVIGATE, LOCATE_ROOM, SOS
■	intent.py — détection intention	Sentence-BERT encode la commande → similarité cosinus → retourne l'intention
■	Extraire les entités	Depuis la phrase extraire : l'objet ou la pièce mentionnée
■	Brancher dans main.py	Le router appelle intent.py pour chaque commande reçue
■	Tester avec phrases texte	Envoie des phrases test via Swagger : 'trouve mes clés', 'va à la cuisine'

Exemple intents.json :

```
{
  "FIND_OBJECT": ["trouve mes clés", "où sont mes lunettes", "cherche le téléphone"],
  "NAVIGATE": ["va à la cuisine", "amène-moi au salon", "direction chambre"],
  "LOCATE_ROOM": ["où suis-je", "dans quelle pièce je suis"],
  "SOS": ["au secours", "appelle aide", "urgence"]
}
```

■ Critère : 'trouve mes lunettes' → FIND_OBJECT + entité=lunettes

PHASE 5 Navigation

Guider l'utilisateur de pièce en pièce

■	Créer house.json	Définir les pièces et leurs connexions : salon↔couloir↔cuisine...
■	graph.py — charger le graphe	Lire house.json et construire un graphe Python (dict ou networkx)
■	pathfinder.py — algorithme Dijkstra	Entrée : pièce A + pièce B → Sortie : liste de pièces du chemin
■	instructions.py — générer les phrases	Transformer [salon, couloir, cuisine] → ['avancez', 'tournez à droite', 'arrivé']
■	Brancher dans /navigate	Appeler les 3 fonctions en chaîne et retourner les instructions

Exemple house.json :

```
{
  "rooms": ["salon", "couloir", "cuisine", "chambre", "sdb"],
  "connections": [
    ["salon", "couloir"], ["couloir", "cuisine"],
    ["couloir", "chambre"], ["couloir", "sdb"]
  ]
}
```

■ Critère : /navigate?from=salon&to=cuisine retourne ['avancez dans le couloir', 'tournez à gauche']

PHASE 6 Module SOS

Gestion des urgences vocales

■	sos.py — logique d'urgence	Déclenche une alerte : log, message, appel (simulé pour la démo)
■	Brancher dans /sos	Si intent = SOS → appeler sos.py immédiatement
■	Retourner signal vibration forte	La réponse JSON inclut vibration: 'strong' pour l'app Flutter
■ Pour la démo : un simple print + log suffit. Pas besoin d'un vrai appel téléphonique.		

PHASE 7

Application Flutter

Construire les 4 écrans + connexion backend

■	Setup Flutter + dépendances	Ajouter dans pubspec.yaml : http, flutter_tts, speech_to_text, vibration, camera
■	api_service.dart	Classe qui fait les appels HTTP vers ton backend FastAPI
■	stt_service.dart + tts_service.dart	STT = écouter le micro. TTS = lire la réponse à voix haute
■	home_screen.dart	Grand bouton micro → envoie commande → reçoit réponse → TTS la lit
■	navigate_screen.dart	Affiche les étapes du chemin une par une avec TTS
■	sos_screen.dart	Bouton rouge SOS + vibration forte
■	settings_screen.dart	Vitesse TTS, contact d'urgence

■ Critère : l'écran principal écoute → envoie au backend → lit la réponse à voix haute.

PHASE 8

Intégration & Tests

Tout brancher et tester bout en bout

■	Connecter Flutter ↔ FastAPI	L'app envoie la commande vocale au backend via Wi-Fi local
■	Connecter la caméra	L'app envoie des frames au /detect-obstacle en continu
■	Test scénario complet 1	"Trouve mes clés" → STT → NLP → CV → TTS
■	Test scénario complet 2	"Va à la cuisine" → STT → NLP → Navigation → TTS étape par étape
■	Test scénario SOS	"Au secours" → STT → SOS → vibration forte + alert
■	Corriger les bugs	Tester sur vrai téléphone (pas seulement émulateur)

■ Lance le backend sur ton PC et connecte le téléphone au même réseau Wi-Fi.

PHASE 9

Démo & Présentation

Préparer la démonstration finale

■	Préparer 3 scénarios de démo	1) Trouver un objet 2) Naviguer vers une pièce 3) SOS
■	Configurer la maison de démo	house.json avec les pièces de la salle de présentation
■	Faire une vidéo de backup	Si le live plante, tu montres la vidéo
■	Préparer les slides	Architecture + pipeline + démo + ce que tu as appris

■ Règle d'or : teste la démo 3 fois la veille sur le vrai matériel.

Résumé de l'ordre de développement

